Harvey Mudd College

E102 Final Project

Project 1: Inverted Pendulum

By Kate Lam and Abraham Rock

October 12, 2026

Background

Overview In this project, we designed a control system for an inverted pendulum stationed on top of a cart. The goal of the problem is to stabilize the pendulum in its upright position while moving the cart to a targeted position of 1 meter. The constraints on the system include no overshoot in the cart's position after 10 seconds, and keeping cart's acceleration under a magnitude of $0.5\frac{\text{m}}{\text{sec}^2}$. (See Appendix 4.2) To accomplish this, we represented the system in state-space. From there, we designed a feedback control using strategic pole placements, augmenting the system with integral control to eliminate steady-state error. Additionally, an observer was added to estimate system states from the outputs. This project demonstrates the efficiency and limitations of modern control theory. (See Appendix 4.3)

1 State Space Design of a Linear Controller for a Linearized Plant

Using small angle approximations of $\sin(\theta) \approx \theta$ and $\cos(\theta) \approx 1$, we can linearize our equations of motion and solve for the highest order derivative terms ($\ddot{\theta}(t)$ and, $\ddot{s}(t)$) for our state-space representation.

$$L\ddot{\theta}(t) - g\sin(\theta(t)) = -a(t)\cos(\theta(t)) + L\alpha(t)$$

$$L\ddot{\theta}(t) - g\theta(t) = -a(t) + L\alpha(t)$$

$$\rightarrow \ddot{\theta}(t) = \alpha(t) + \frac{g}{L}\theta(t) - \frac{1}{L}a(t)$$

$$\rightarrow \ddot{s}(t) = a(t)$$

With state vector $\bar{\mathbf{x}} = \begin{bmatrix} \theta(t) \\ \dot{\theta}(t) \\ s(t) \\ \dot{s}(t) \end{bmatrix}$, $\dot{\bar{\mathbf{x}}}$ becomes $= \begin{bmatrix} \dot{\theta}(t) \\ \ddot{\theta}(t) \\ \dot{s}(t) \\ \ddot{s}(t) \end{bmatrix}$.

So by inspection, we can develop our Non-Canonical Factor Form Matrices A, B, C, and D with our governing equations coefficients in the form of our state vectors.

$$\begin{bmatrix} \dot{\theta}(t) \\ \ddot{\theta}(t) \\ \dot{s}(t) \\ \ddot{s}(t) \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ \frac{g}{L} & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} \theta(t) \\ \dot{\theta}(t) \\ s(t) \\ \dot{s}(t) \end{bmatrix} + \begin{bmatrix} 0 \\ -\frac{1}{L} \\ 0 \\ 1 \end{bmatrix} a(t) + \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix} \alpha(t)$$

$$\begin{bmatrix} \theta(t) \\ s(t) \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} \theta(t) \\ \dot{\theta}(t) \\ s(t) \\ \dot{s}(t) \end{bmatrix} + \begin{bmatrix} 0 & 0 \end{bmatrix} a(t)$$

With our state space model constructed, we next calculated our M_C and M_O matrices to verify that our system is controllable and observable (See computation in Appendix 4.1). With a 4×4 M_C and, 8×4 M_O, both

have the rank equal to the number of states which is 4, confirming that all states can be controlled and estimated from our inputs and outputs.

For pole placements, we approximate the dominant behavior as a second-order system, selecting a damping ratio ζ, and natural frequency ω_n. The relationship between the two is calculated with settling time where $t_s = \frac{-\ln(0.01)}{\zeta\omega_n}$ to ensure settling time of 1% within 10 seconds and how fast our system should be for a critically damped or over damped system to have no overshoot in cart position. With second-order canonical form, we can use these parameters to place two conjugate poles $s^2 + 2\zeta\omega_n s + \omega_n^2 \rightarrow \frac{-2\zeta\omega_n \pm \sqrt{4\zeta^2\omega_n^2 - 4\omega_n^2}}{2}$. Since our 4 state system expects 4 poles, we can place another two conjugate poles $10\times$ faster to ensure we keep our two initial ones as dominant. To utilize integral action, we augment the system with a 5th state and place its pole at $20\times$ the real part of the slower dominant pole. Since complex poles must appear in conjugate pairs, we use only the real part here.

Using MATLAB's place() function, we can input these desired poles to find the controller's $K_a = [K_I, k_1, k_2, k_3, k_4]$. K_I is the controller associated with integral action, and is found by multiplying the first index of K_a by -1 since K_I is a positive gain as opposed to the negative feedback gain $\bar{\mathbf{K}}$. We place observer poles $100\times$ faster of our same initial two conjugate pairs of poles to ensure the observer converges significantly faster than the controller. (See Appendix 4.1)

2 Simulation of Control of Nonlinear Plant with the Linear Controller

With our controller and observer designed (Appendix 4.3), we can test our pole placement and controller gains in Simulink with the original nonlinear system to see our transient response.

While our initial pole guesses produced a stable looking system, our input acceleration was fully in saturation indicating our controller was demanding more than our saturated input could handle. Additionally, small angle approximation falls apart under the large pendulum deviation. This prevented the cart from settling at 1 meter since the controller had lost *control* of the system. (1)

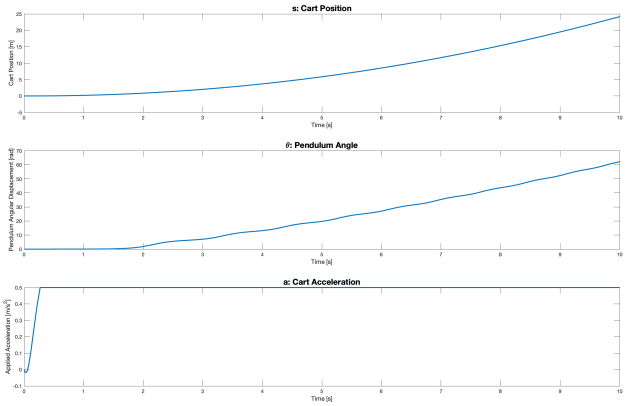


Figure 1: Response of $s, \theta,$ and, a with Initial Pole Guesses

To improve this response, we looked at our aug-

mented feedback matrix A_{af}'s eigenvectors. Since K_I drives the integral control responsible for achieving the 1 meter position steady state, we focused on increasing the eigenvector in K_a that influences K_I the most. By identifying which pole corresponds to which eigenvector via the eigenvalues of A_{af}, we can selectively increase or decrease a pole's magnitude to produce a larger K_I while adjusting parameters ζ and ω_n along with k_2 and k_4 which control the angular velocity of the pendulum and velocity of the cart to ensure we move slow enough to control the positions.

To improve robustness to disturbance α, we added a small initial condition to our Observer's integrator. Adding $\begin{bmatrix} 0 & \frac{\alpha}{10} & 0 & \frac{\alpha}{10} \end{bmatrix}$ gives the integrator a starting point that isn't 0 and quickly handles disturbances by assuming we will already have some cart and pendulum velocities.

3 Discussion and Conclusions

Our system parameter values we picked after tuning are ($\zeta = 3,\ \omega_n = 0.62$) $\therefore K_I = -24.23$ Looking at the value of the cart's position at 10 seconds, our cart is at 0.9955 meters, and approaching a steady state of $0.\overline{99} \approx 1.0$

Now as we increment our angular acceleration disturbance α, we start to see a fault in our system. We tuned our pole placement expecting a 0.5 disturbance, so that means any smaller or larger disturbance results in our controller failing to control the system. Once we reach a disturbance of $\alpha = 0.62 \frac{\text{rad}}{\text{sec}^2}$ on the pendulum, the cart position overshoots the 1-meter constraint.

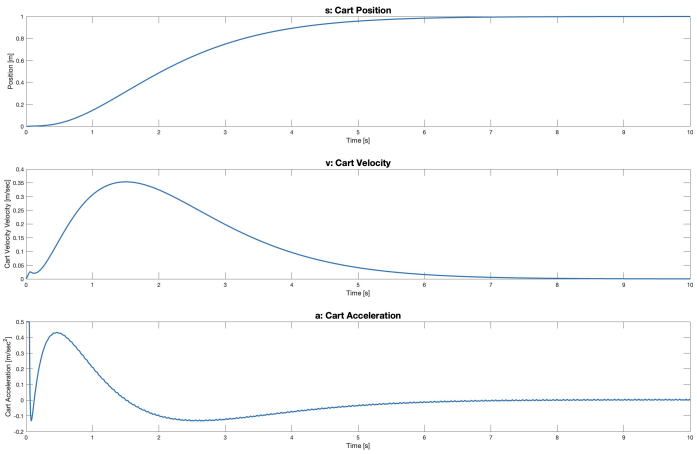


Figure 2: Response of s, ṡ, and, a after fine tuning poles

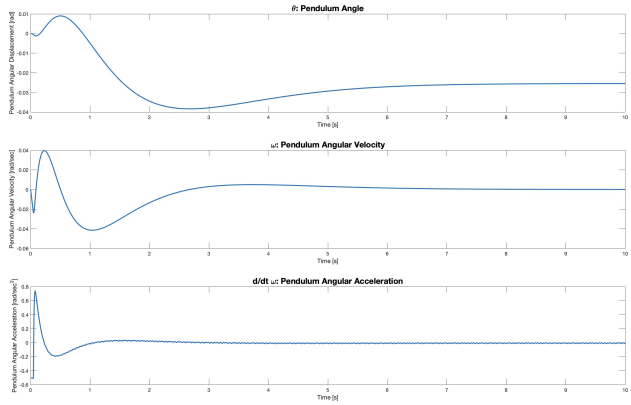


Figure 3: Response of $\theta, \dot{\theta}$, and $\ddot{\theta}$ after fine tuning poles

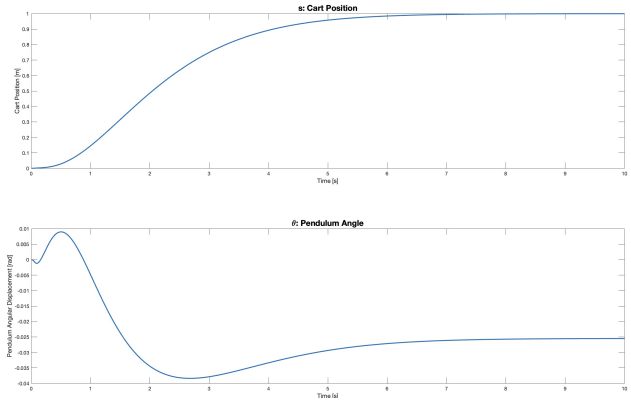


Figure 4: Outputs of System, s and θ

When we plotted our final cart states (Figure 2), pendulum states (Figure 3), and outputs (Figure 4), we saw a pattern that developed between theta and acceleration. In its initial stages, acceleration goes up and rails out at $0.5\frac{m}{s}$ only to go back down and up a little again before oscillations is introduced into acceleration after 2 seconds. We see a similar shape with theta's plot, first curving up, then back down, then up and steadying, but no oscillations. The beginning increase in acceleration is due to us placing our poles aggressively, so the system tries to correct to the desired pole quickly, but cannot, so acceleration is saturated.

To keep the pendulum upright and its theta angle close to zero degrees, we will have to "oscillate" very quickly in different directions to make sure the pendulum doesn't tilt down. This is what the cart does once it reaches a point of relative stability, it oscillates very little to accommodate the inverted pendulum from tilting, especially when we increase our disturbances.

We explored linear quadratic regulator (LQR) control to help us strategically choose controller gains. We design a desired Q and R matrix that takes care of state weighting and control weighting, respectively. According to Bryson's rule, Q is a diagonal matrix that penalizes state errors corresponding to the weight of the state. Δx_i^2 is indicative of maximum acceptable error, which we want minimized.

R is also a diagonal matrix that penalizes actuator usage. Increasing the Q matrix would result in faster control responses that minimize state error quicker, but it ends up being noisier. Having a low Q and high R, means that our system approaches steady state slower and less precise, but reduces wear on the actuator. We made the velocity terms larger in order to ensure we have more control over our desired positions. This helps make our system more robust too, ensuring we are stable for disturbances up to $w \leq 0.93 \frac{\text{rad}}{\text{sec}^2}$.

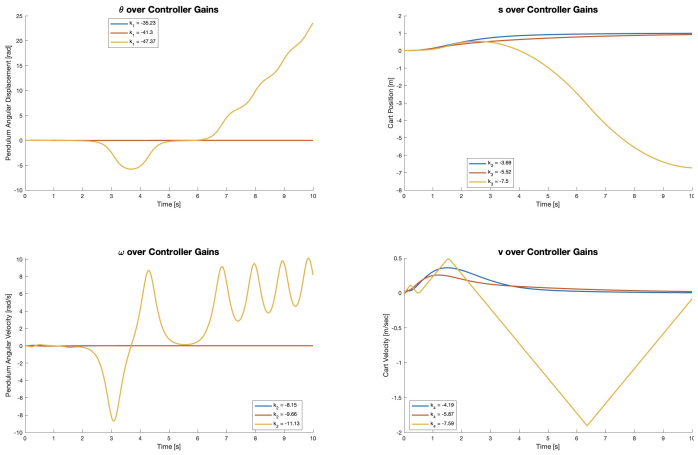


Figure 5: Response of internal states over varied Controller Gains

In Figure 5 we can see varying $\overline{K}$, can vary the response of our states, even to the extent of making our system unstable.

Lastly, one thing we noticed was the faster our observer poles, the more noise we amplified, making our acceleration input react to unnecessary small signals. In the future I would be interested in applying a low-pass filter to remove any large outliers in the input to the Observer to be able to safely have fast observer poles.

4 Appendices

4.1 Code

```
%% Controller and Observer System
% Constants
PendulumLength = 0.5;   % meters
alpha = 0.5;            % rad/sec^2
g = 9.81;               % m/sec^2

% State Space Equation Matrices
A = [0, 1, 0, 0; g/PendulumLength, 0, 0, 0; 0, 0, 0, 1; 0, 0, 0, 0];
B = [0; -1/PendulumLength; 0; 1];
Bw = [0; 1; 0; 0];
Cwhole = [1, 0, 0, 0; 0, 0, 1, 0];
C = Cwhole(2,:); % Interested in position of cart
D = 0;

% Augmented State Space Matrices
Aa = [zeros(height(C), 1), -C; zeros(height(A), 1), A];
Ba = [zeros(height(C), 1); B];
Ca = [0, C];
Da = 0;
sys_a = ss(Aa, Ba, Ca, Da);

%% NCF to CCF and OCF
% Controllability Matrix
Mc = [B, A*B, A*A*B, A*A*A*B];

```

```
% Observabilty Matrix
Mo = [Cwhole; Cwhole*A; Cwhole*A*A; Cwhole*A*A*A];

% Transformation Matrices
tnc = [0, 0, 0, 1]/Mc;
Tc = inv([tnc*A*A*A; tnc*A*A; tnc*A; tnc]);
A_CCF = Tc\A*Tc;

%% Controller and Observer Poles
% Desired System Parameters
zeta = 3;
w_n = 1.4;
ts = -log(0.001)/(zeta*w_n);

pole = flip(roots([1, 2*zeta*w_n, w_n^2])');
poles = [2.9*pole, 3.5*pole, 5*real(pole(1))];

Q = diag([11, 7, 8, 22, 16]);
R = 5;

% Controller Poles
%Ka = place(Aa, Ba, poles);
[Ka, S, P] = lqr(sys_a, Q, R);
KI = -Ka(1);
KI_SIM = Ka(1);
K = Ka(2:end);

% Observer Poles
```

```
L = place(A', Cwhole', 100*P(1:4))';
D_obs = [0; 0];

% Augmented Feedback Matrices
Aaf = Aa - Ba*Ka;
Baf = [1; zeros(height(A), 1)];
Caf = [D*KI, C];
Daf = 0;

%% Run the Simulation
tstop = 10;
sim_output = sim('project1_sim', tstop);

% Output Signals
a = sim_output.yout(:, 1);
theta = sim_output.yout(:, 2);
s = sim_output.yout(:, 3);

% Internal Signals
t = sim_output.tout;
omega = sim_output.logsout{1}.Values.Data;      % Angular Velocity
s_dot = sim_output.logsout{2}.Values.Data;      % Cart Velocity
omega_dot = sim_output.logsout{3}.Values.Data;  % Angular Acceleration
w = sim_output.logsout{4}.Values.Data;          % Disturbance
a_fb = sim_output.logsout{5}.Values.Data;       % Feedback input
a_I = sim_output.logsout{6}.Values.Data;        % Integral Gain (error)
x_I = sim_output.logsout{7}.Values.Data;        % Integral of Error
xdot_I = sim_output.logsout{8}.Values.Data;     % Error
```

```
a_des = sim_output.logsout{9}.Values.Data;      % Desired Acceleration
   Gain

% Plotting 1
figure(1)
hold on
subplot(2, 1, 1)
plot(t, s, lineWidth=2);
xlabel("Time [s]", 'FontSize', 12)
ylabel("Cart Position [m]", 'FontSize', 12)
title("s: Cart Position", 'FontSize', 16)

subplot(2, 1, 2)
plot(t, theta, lineWidth=2);
xlabel("Time [s]", 'FontSize', 12)
ylabel("Pendulum Angular Displacement [rad]", 'FontSize', 12)
title("\theta: Pendulum Angle", 'FontSize', 16)
hold off

% Plotting 2
figure(2)
hold on
subplot(3, 1, 1)
plot(t, theta, lineWidth=2)
xlabel("Time [s]", 'FontSize', 12)
ylabel("Pendulum Angular Displacement [rad]", 'FontSize', 12)
title("\theta: Pendulum Angle", 'FontSize', 16)

```

```
subplot(3, 1, 2)
plot(t, omega, lineWidth=2)
xlabel("Time [s]", 'FontSize', 12)
ylabel("Pendulum Angular Velocity [rad/sec]", 'FontSize', 12)
title("\omega: Pendulum Angular Velocity", 'FontSize', 16)

subplot(3, 1, 3)
plot(t, omega_dot, lineWidth=2)
xlabel("Time [s]", 'FontSize', 12)
ylabel("Pendulum Angular Acceleration [rad/sec^2]", 'FontSize', 12)
title("d/dt \omega: Pendulum Angular Acceleration", 'FontSize', 16)
hold off

% Plotting 3
figure(3)
hold on
subplot(3, 1, 1)
plot(t, s, lineWidth=2)
xlabel("Time [s]", 'FontSize', 12)
ylabel("Position [m]", 'FontSize', 12)
title("s: Cart Position", 'FontSize', 16)

subplot(3, 1, 2)
plot(t, s_dot, lineWidth=2)
xlabel("Time [s]", 'FontSize', 12)
ylabel("Cart Velocity Velocity [m/sec]", 'FontSize', 12)
title("v: Cart Velocity", 'FontSize', 16)

```

```
subplot(3, 1, 3)
plot(t, a, lineWidth=2)
xlabel("Time [s]", 'FontSize', 12)
ylabel("Cart Acceleration [m/sec^2]", 'FontSize', 12)
title("a: Cart Acceleration", 'FontSize', 16)
hold off

legend_theta = strings(1, 3);
legend_s = strings(1, 3);
legend_omega = strings(1, 3);
legend_s_dot = strings(1, 3);

tK = cell(1, 3);
thetaK = cell(1, 3);
sK = cell(1, 3);
omegaK = cell(1, 3);
s_dotK = cell(1, 3);

for idx = 1:3
    Q = diag([11, idx*7, 8, (idx)*22, (idx/2)*16]);
    R = 5;
    [Ka, S, P] = lqr(sys_a, Q, R);
    KI = -Ka(1);
    KI_SIM = Ka(1);
    K = Ka(2:end);
    L = place(A', Cwhole', 100*P(1:4))';

    sim_output = sim('project1_sim', tstop);
```

```
165
166     tK{idx} = sim_output.tout;
167     thetaK{idx} = sim_output.yout(:, 2);
168     sK{idx} = sim_output.yout(:, 3);
169     omegaK{idx} = sim_output.logsout{1}.Values.Data;
170     s_dotK{idx} = sim_output.logsout{2}.Values.Data;
171
172     legend_theta(idx) = "k_1 = " + num2str(round(K(1), 2));
173     legend_omega(idx) = "k_2 = " + num2str(round(K(2), 2));
174     legend_s(idx) = "k_3 = " + num2str(round(K(3), 2));
175     legend_s_dot(idx) = "k_4 = " + num2str(round(K(4), 2));
176 end
177
178 % Plotting 4 (so so gross)
179 figure(4)
180 subplot(2, 2, 1)
181 hold on
182 for idx = 1:3
183     plot(tK{idx}, thetaK{idx}, 'LineWidth', 2)
184 end
185 xlabel("Time [s]", 'FontSize', 12)
186 ylabel("Pendulum Angular Displacement [rad]", 'FontSize', 12)
187 title("\theta over Controller Gains", 'FontSize', 16)
188 legend(legend_theta);
189 hold off
190
191 subplot(2, 2, 2)
192 hold on
```

```
for idx = 1:3
    plot(tK{idx}, sK{idx}, 'LineWidth', 2)
end
xlabel("Time [s]", 'FontSize', 12)
ylabel("Cart Position [m]", 'FontSize', 12)
title("s over Controller Gains", 'FontSize', 16)
legend(legend_s);
hold off

subplot(2, 2, 3)
hold on
for idx = 1:3
    plot(tK{idx}, omegaK{idx}, 'LineWidth', 2)
end
xlabel("Time [s]", 'FontSize', 12)
ylabel("Pendulum Angular Velocity [rad/s]", 'FontSize', 12)
title("\omega over Controller Gains", 'FontSize', 16)
legend(legend_omega);
hold off

subplot(2, 2, 4)
hold on
for idx = 1:3
    plot(tK{idx}, s_dotK{idx}, 'LineWidth', 2)
end
xlabel("Time [s]", 'FontSize', 12)
ylabel("Cart Velocity [m/sec]", 'FontSize', 12)
title("v over Controller Gains", 'FontSize', 16)
```

```
legend(legend_s_dot);
hold off
```

4.2 Problem Statement

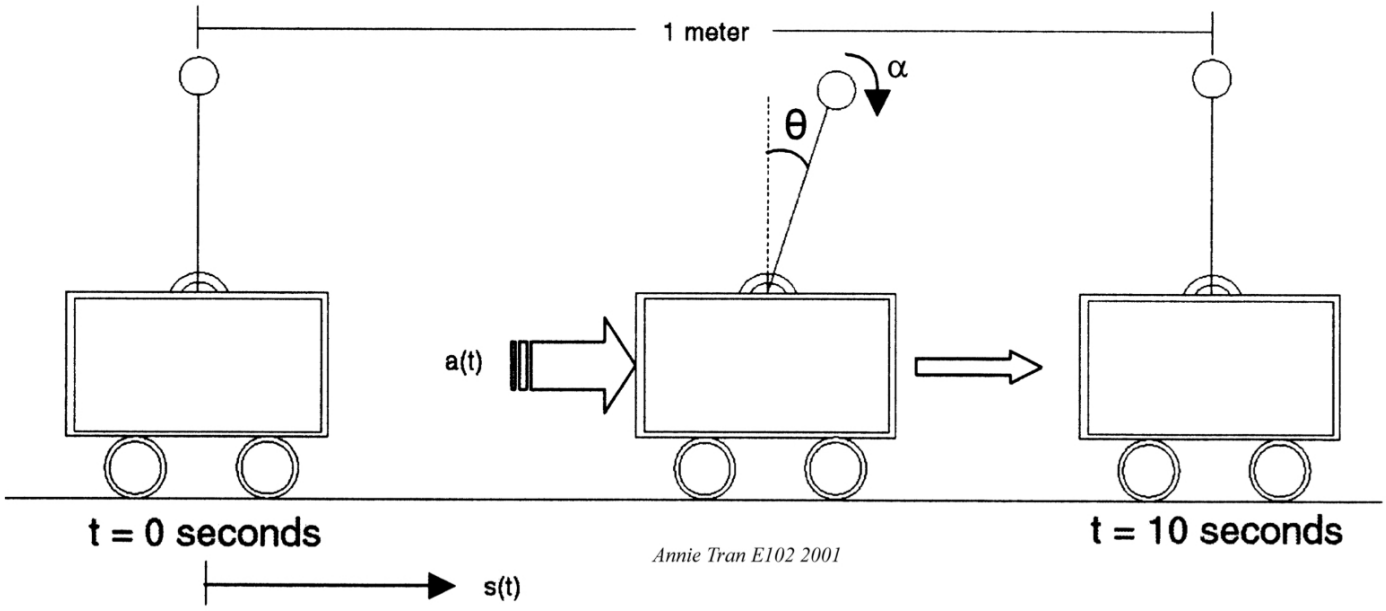


Figure 6: Pendulum Model

4.3 Simulink Models

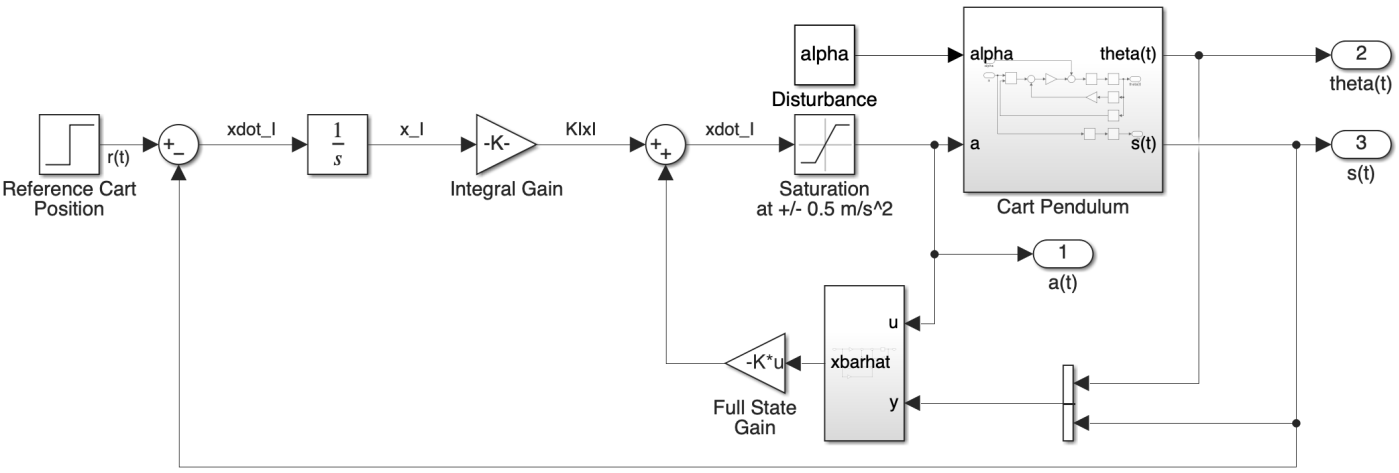


Figure 7: Simulink Model

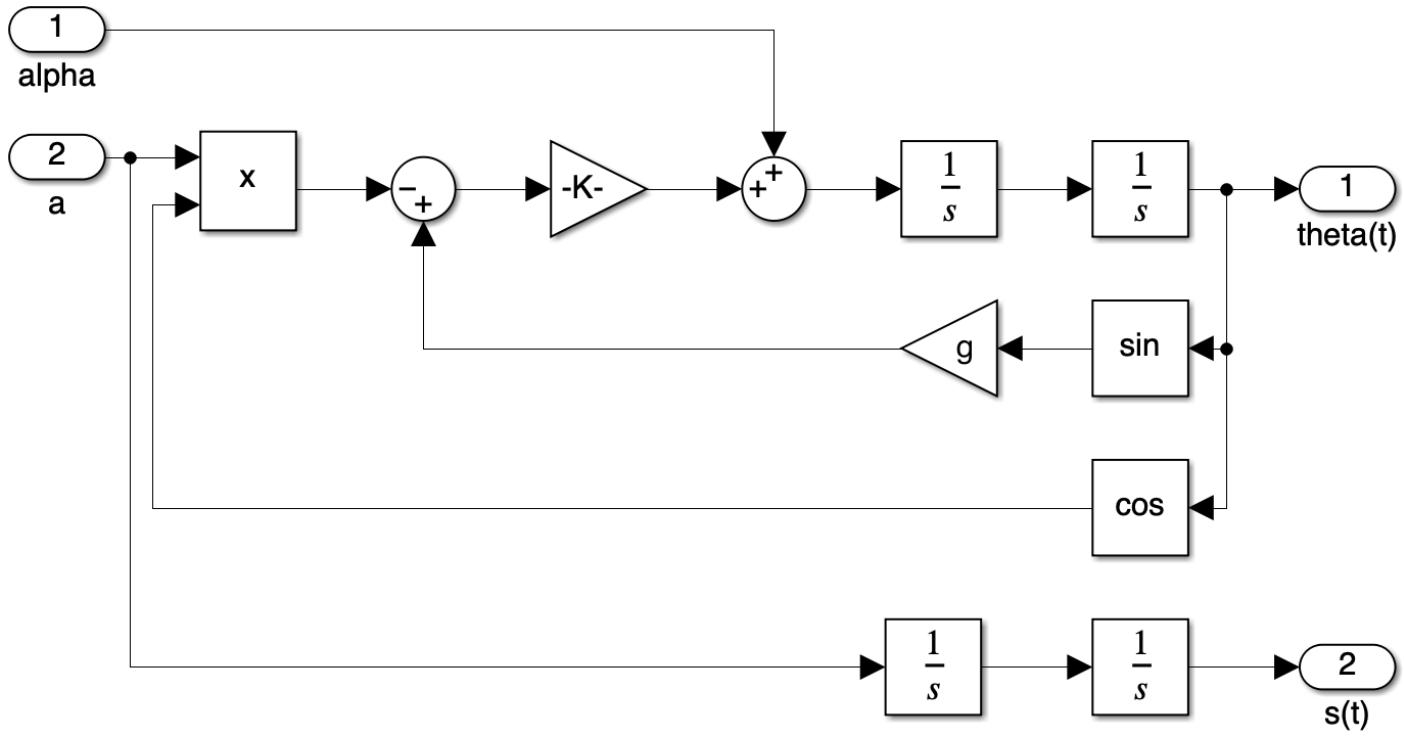


Figure 8: Simulink System of the Cart Pendulum

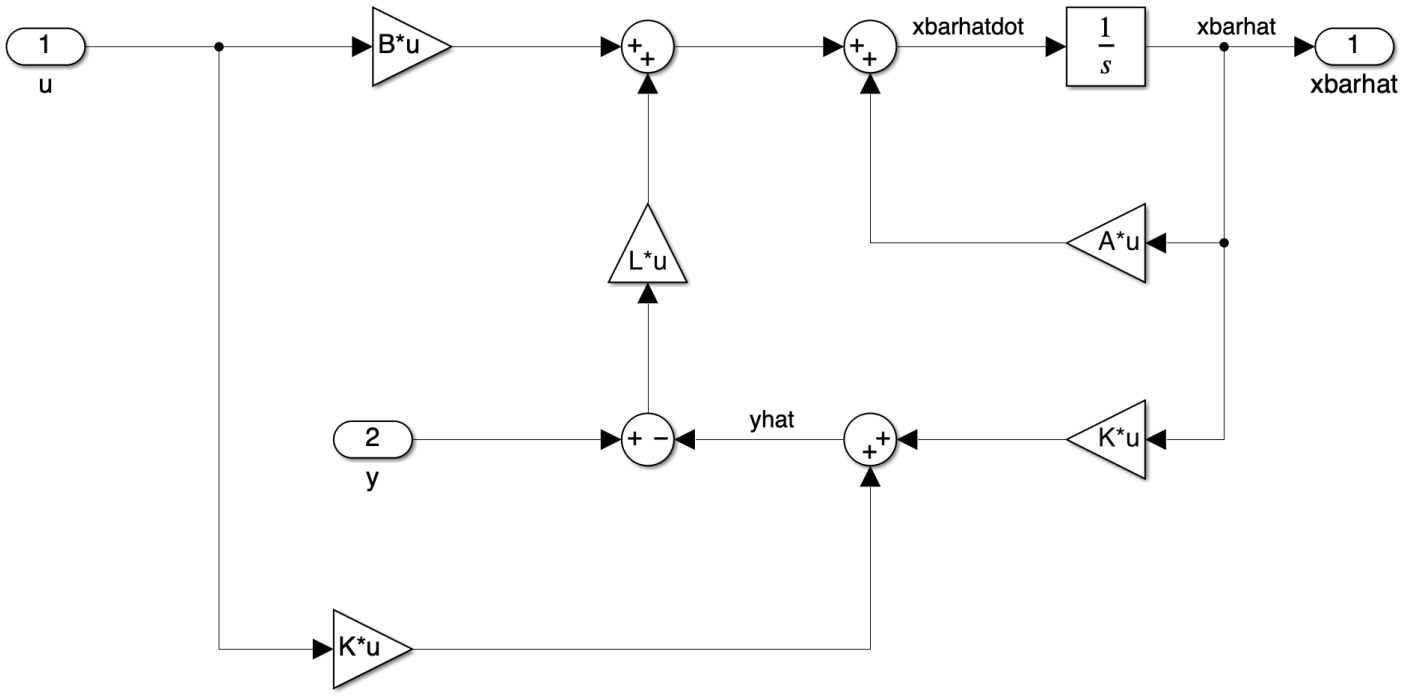


Figure 9: Simulink Model of Observer